DEPARTMENT OF MECHANICAL ENGINEERING

ABES EC/ AKTU, LUCKNOW

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: Smart Traffic Light Control System

Standard Academic Compliance: IEEE Std 830-1998

SUBMITTED BY:	Project Team
Degree:	Bachelor of Technology (B.Tech)
Academic Year:	2026 - 2027
UNDER THE GUIDANCE OF:	Prof. / Dr. _____
Designation / Title:	_____
Department:	Electronics & Communication / Project Department
Status:	3rd Year Project Specification
Date of Submission:	DD-MM-YYYY

IoT-Based Adaptive Traffic Signal Management using STM32, Wokwi, MQTT and Cloud Dashboard

Page Reference - IEEE Std 830-1998 Academic Template

Document Revision & Approval History

Version	Date	Description / Major Changes	Prepared By	Reviewed By
0.1	01-Oct-2026	Initial Draft & Scope Definition	Project Team	Project Guide
1.0	01-Oct-2026	Final 3rd Year Project SRS Baseline	Project Team	HOD / Committee

1. Introduction

1.1 Purpose

This SRS defines the functionality, external interfaces, performance targets, constraints and verification approach for the Smart Traffic Light Control System. The embedded target is STM32; the networked Wokwi demonstration uses ESP32 as a Wi-Fi/MQTT gateway.

1.2 Scope of the System

The system combines ultrasonic traffic sensing, adaptive signal control, 7-segment countdown, LCD status, keypad/PIN and fingerprint-access abstraction, buzzer alerts, configuration persistence, MQTT telemetry, PostgreSQL logging and a live Render-hosted dashboard.

1.3 Definitions, Acronyms, and Abbreviations

Category	Term / Acronym	Definition / Standard Context
Standard	SRS	Software Requirements Specification (IEEE Std 830-1998 format).
Embedded	STM32	ARM microcontroller family used as intended traffic controller.
IoT	MQTT	Publish/subscribe messaging protocol.
Database	PostgreSQL	Relational database for telemetry.
Security	PIN / Fingerprint	Two access mechanisms; fingerprint is abstracted in simulation.

2. Overall Description

2.1 Product Perspective

The system has embedded control, local HMI/security, MQTT messaging, dashboard application and PostgreSQL storage layers. Wokwi currently does not support multiple microcontrollers in one simulation, so the networked demonstration uses ESP32 separately from the STM32 target firmware.

2.2 User Classes and Characteristics

User Role / Persona	Technical Proficiency	Core Responsibilities & Rights
System Administrator	High	Deployment, database, MQTT and system configuration.
Traffic Operator / Registered User	Intermediate	Monitoring, logs and authorized signal control.
Guest / Public Visitor	Basic	Read-only dashboard overview.

2.3 Operating Environment

STM32F103C8 target; Wokwi ESP32 network simulation; Node.js/Express on Render; PostgreSQL; MQTT broker; modern web browser.

2.4 Design and Implementation Constraints

Free/academic cloud resources, secrets stored in environment variables, safe signal states, documented Wokwi limitations, fingerprint abstraction, and optional Bluetooth interface.

2.5 Assumptions and Dependencies

Stable sensor inputs, regulated supply, MQTT availability, Render/PostgreSQL availability and working browser/network.

3. System Features and Functional Requirements

3.1 Traffic Sensing and Adaptive Signal Control

Req ID	Feature	Description	Input / Validation Rule	Priority
FR-1.1	Traffic Data Acquisition	Read ultrasonic distance and estimate density.	Valid 2–400 cm; timeout invalid.	High
FR-1.2	Adaptive Signal Timing	Select phase and green duration based on demand.	Minimum/maximum timing bounds.	High
FR-1.3	Signal Safety	Prevent conflicting green outputs.	At most one direction GREEN.	Critical
FR-1.4	Safe Fallback	Use deterministic phase if input invalid.	Sensor timeout handling.	High

3.2 HMI, Security and Configuration

Req ID	Feature	Description	Input / Validation Rule	Priority
FR-2.1	7-Segment Timer	Display remaining signal time.	00–99 s.	High
FR-2.2	LCD Status	Display distances, phase and mode.	Refresh from latest readings.	Medium
FR-2.3	Keypad Access	Accept PIN for protected operations.	Four-digit PIN.	High
FR-2.4	Fingerprint Access	Provide biometric authorization interface.	Simulation uses success input.	High
FR-2.5	Buzzer Alert	Signal authentication/safety events.	Short audible feedback.	Medium
FR-2.6	Configuration Storage	Persist timing/configuration.	Non-volatile storage.	Medium

3.3 IoT Telemetry, Dashboard & Administration

Req ID	Feature	Description	Input / Validation Rule	Priority
FR-3.1	MQTT Telemetry	Publish JSON traffic state.	Topic/broker configurable.	High
FR-3.2	Database Logging	Persist received samples.	Relational telemetry schema.	High
FR-3.3	Live Dashboard	Show distances, signals, timer and mode.	Live event update.	High
FR-3.4	Historical Log	Display recent telemetry.	Newest-first bounded query.	Medium
FR-3.5	Remote Control	Publish operator mode commands.	AUTO/MANUAL commands.	Medium
FR-3.6	Health Monitoring	Expose service/MQTT/DB status.	/health endpoint.	Medium

4. External Interface Requirements

4.1 User Interfaces

Responsive web dashboard with live KPI cards, signal indicators, density bars, telemetry table and remote-control buttons. Embedded HMI uses LCD, 7-segment countdown, signal LEDs and buzzer.

4.2 Hardware Interfaces

Ultrasonic TRIG/ECHO; six signal LEDs; TM1637 7-segment; I²C LCD; matrix keypad; UART fingerprint interface in final hardware; buzzer; non-volatile configuration storage.

4.3 Software Interfaces

Node.js/Express, MQTT client and PostgreSQL. JSON is used for telemetry serialization.

4.4 Communication Interfaces

MQTT for IoT telemetry/control, HTTP/HTTPS for web APIs and SSE for live browser updates.

5. Non-Functional Requirements (NFRs)

Attribute	Target Metric / Standard	Verification
Performance	Normal API queries < 500 ms target; 2 s demo telemetry.	API/browser timing.
Security	No secrets in Git; parameterized SQL.	Configuration/code review.
Reliability	MQTT reconnect; safe local operation offline.	Disconnect/reconnect test.
Portability	Responsive browser and Render/GitHub deployment.	Multi-browser/deployment test.
Maintainability	Separated firmware, server, DB and UI.	Repository/code review.

6. System Design and Analysis Models

6.1 System Context / Architecture

External Entity	Information Flow	System Boundary
Ultrasonic Sensors	Distance / density	STM32 Traffic Controller
Keypad / Fingerprint	Authentication / mode request	Security & Access Module
Controller Outputs	R/Y/G, buzzer, timer, LCD	Local HMI
IoT Gateway	JSON telemetry / commands	MQTT Layer
Dashboard	Live monitoring / control	Web Application
Database	Telemetry persistence	PostgreSQL

6.2 Data Flow Diagram

Level 0: Sensors/Security → Traffic Control Engine → Signal/HMI. Traffic Control Engine → MQTT Telemetry → Dashboard Backend → PostgreSQL. Operator Dashboard → MQTT Control → Traffic Controller.

6.3 Entity-Relationship Model

Entity	Primary Key	Important Attributes	Relationship
traffic_telemetry	id	device_id, ts, north_cm, east_cm, counts, signals, timer, mode, access flag, raw JSON.	Many records per device.
Device (logical)	device_id	Device/deployment identifier.	1-to-many telemetry.
MQTT Topic (logical)	topic	Telemetry/control route.	Maps message direction.

6.4 Activity / Sequence Model

Sensor Read → Estimate Density → Select Phase → Update Signal Outputs → Update Timer/LCD → Publish MQTT JSON → Backend Receives → Store PostgreSQL → Push Live Dashboard Update. If MQTT is unavailable, local signal operation continues and the network connection is retried.

6.5 Deployment Model

Layer	Component	Deployment Location
Embedded	STM32 traffic controller	Target hardware / development board
Simulation Gateway	ESP32 network prototype	Wokwi
Messaging	MQTT Broker	Public/managed broker
Application	Node.js/Express Dashboard	Render Web Service
Persistence	PostgreSQL	Render PostgreSQL
Source Control	Git repository	GitHub

7. Academic Review & Sign-Off

Project Coordinator / Guide	Internal Examiner	Head of Department (HOD)
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Implementation Deliverables

1. Wokwi simulation: wokwi/. 2. STM32 target-reference firmware: stm32-reference/. 3. MQTT/PostgreSQL dashboard: server/ and database/. 4. Render deployment: render.yaml. 5. Documentation/report: docs/.

Simulation note: The supplied report preserves the STM32 architecture while separating the ESP32 network demonstration because Wokwi currently does not support multiple microcontrollers in a single simulation.